

BSG INSTITUTE

Sistema de Gestión de la Calidad — RE-I&D-041

Profesor: Ing. Andrés Felipe Rojas Parra

Curso: Diseño de Infraestructura Escalable

INFRAESTRUCTURA CLOUD PARA LLMs

Caso Práctico Completo: Despliegue de un Chatbot Empresarial

Plataforma Seleccionada: Microsoft Azure

Seis Capítulos — 12 Sesiones de 2.5 horas

Arquitecto de Soluciones Senior | AWS · Azure · GCP

Por qué Azure para este curso

Microsoft Azure es la plataforma cloud seleccionada por tres razones fundamentales:

- Azure for Students ofrece USD \$100 de crédito gratuito sin requerir tarjeta de crédito, eliminando la barrera de entrada que frecuentemente ocurre con AWS.
- Azure OpenAI Service proporciona acceso a modelos GPT-4o y GPT-4 con SLA empresarial, ideal para simular entornos de producción reales en clase.
- La documentación en español de Azure es la más completa del mercado hispanoablante, facilitando el autoaprendizaje del estudiante fuera de clase.

El Caso Práctico: Chatbot de Soporte Técnico Empresarial

A lo largo de los seis capítulos construiremos un sistema real y funcional: un Chatbot de Soporte Técnico para la empresa ficticia TechCorp Latinoamérica, una compañía de manufactura con 3,000 empleados.

Característica	Descripción
Problema	El equipo de IT recibe 500 tickets/día. El 60% son preguntas repetitivas sobre configuración de VPN, instalación de software y errores comunes.
Solución	Chatbot conversacional con Azure OpenAI (GPT-4o) que consulta la base de conocimiento interna y responde automáticamente.
SLA requerido	99.9% de disponibilidad, latencia < 2s en el percentil 95.
Volumen pico	1,000 conversaciones simultáneas en horario de oficina (8am—12pm).
Presupuesto	USD \$2,500 / mes para infraestructura cloud.

Capítulo 1: Introducción al Diseño de Infraestructura para LLMs

Dominio cognitivo: Entender | Sesiones 1.1 y 1.2 | 5 horas totales

Sesión 1.1 — Introducción al Curso y Conceptos Básicos

Requisitos de infraestructura para LLMs

Un Large Language Model en producción no es simplemente un modelo de machine learning. Es un sistema distribuido con múltiples capas de infraestructura. Para nuestro chatbot TechCorp los requisitos son:

- **Cómputo:** GPUs o CPUs optimizadas. Azure OpenAI Service gestiona esto transparentemente; nosotros solo consumimos el API.
- **Red:** Latencia baja entre el frontend (navegador del empleado) y el backend (Azure OpenAI). Usaremos Azure Front Door para distribución geográfica.
- **Almacenamiento:** Vectores para la base de conocimiento (Azure AI Search) y logs de conversación (Azure Cosmos DB).
- **Seguridad:** Identidades gestionadas (Managed Identity), sin credenciales en código.

Comparativa de plataformas cloud para LLMs

Criterio	Azure	GCP	AWS
Cuenta gratuita estudiantes	USD \$100, sin tarjeta	USD \$300, con tarjeta	USD \$50–100, con tarjeta
LLM gestionado	Azure OpenAI (GPT-4o)	Vertex AI (Gemini)	Amazon Bedrock
Vector DB nativo	Azure AI Search	AlloyDB / Vertex	OpenSearch
Serverless LLM	Azure Functions + APIM	Cloud Functions	Lambda + API GW
Documentación en español	Muy completa	Parcial	Parcial

Sesión 1.2 — Casos de Uso Empresariales

Arquitectura de referencia del caso TechCorp

La arquitectura que construiremos sigue el patrón RAG (Retrieval-Augmented Generation):

- **Capa de presentación:** Aplicación web React alojada en Azure Static Web Apps.
- **API Gateway:** Azure API Management (APIM) gestiona autenticación, throttling y facturación por usuario.
- **Lógica de negocio:** Azure Functions (Python) orquesta el flujo RAG.
- **Recuperación de conocimiento:** Azure AI Search con índices vectoriales sobre manuales PDF del área IT.
- **Modelo LLM:** Azure OpenAI Service (GPT-4o, deployment regional en East US 2).
- **Persistencia:** Azure Cosmos DB para historial de conversaciones.
- **Observabilidad:** Azure Monitor + Application Insights.

Actividad 1.2: Los estudiantes mapean sus propios requisitos sobre esta arquitectura. Cada grupo elige un sector (banca, salud, manufactura) y documenta SLA, volumen y restricciones de datos.

Capítulo 2: Sistemas Distribuidos y Contenedores

Dominio cognitivo: Aplicar | Sesiones 2.1 y 2.2 | 5 horas totales

Sesión 2.1 — Kubernetes y Docker en Azure

Conceptos fundamentales aplicados al caso TechCorp

La Azure Function que orquesta el flujo RAG se dockeriza para garantizar reproducibilidad entre desarrollo, staging y producción.

Paso 1. Prerrequisitos (ejecutar en terminal local)

```
# Instalar Azure CLI
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az login --use-device-code

# Instalar kubectl y kubelogin
az aks install-cli
```

Paso 2. Crear el grupo de recursos base

```
az group create \
  --name rg-techcorp-llm \
  --location eastus2
```

Paso 3. Crear Azure Container Registry (ACR)

```
az acr create \
  --resource-group rg-techcorp-llm \
  --name acrtechcorp \
  --sku Basic \
  --admin-enabled true
```

Paso 4. Dockerizar la Azure Function RAG

Estructura del proyecto:

```
techcorp-rag/
├─ Dockerfile
├─ requirements.txt
└─ function_app.py
```

Contenido del Dockerfile:

```
FROM mcr.microsoft.com/azure-functions/python:4-python3.11
ENV AzureWebJobsScriptRoot=/home/site/wwwroot \
    AzureFunctionsJobHost__Logging__Console__IsEnabled=true
COPY requirements.txt /home/site/wwwroot/
RUN pip install -r /home/site/wwwroot/requirements.txt
COPY . /home/site/wwwroot
```

Paso 5. Construir y subir la imagen al ACR

```
cd techcorp-rag
az acr build \
  --registry acrtechcorp \
  --image rag-function:v1.0 .
```

Sesión 2.2 — Práctica con Contenedores y AKS

Despliegue en Azure Kubernetes Service

Paso 6. Crear el clúster AKS con nodos optimizados para LLM

```
az aks create \
  --resource-group rg-techcorp-llm \
  --name aks-techcorp \
  --node-count 2 \
  --node-vm-size Standard_D4s_v3 \
  --attach-acr acrtechcorp \
  --enable-managed-identity \
  --generate-ssh-keys
```

Paso 7. Conectar kubectl al clúster

```
az aks get-credentials \
  --resource-group rg-techcorp-llm \
  --name aks-techcorp
kubectl get nodes # Verificar nodos activos
```

Paso 8. Crear el deployment Kubernetes

Archivo rag-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rag-function
spec:
  replicas: 2
  selector:
    matchLabels:
```

```

  app: rag-function
template:
  metadata:
    labels:
      app: rag-function
  spec:
    containers:
      - name: rag-function
        image: acrtechcorp.azurecr.io/rag-function:v1.0
        ports:
          - containerPort: 80
        env:
          - name: AZURE_OPENAI_ENDPOINT
            valueFrom:
              secretKeyRef:
                name: openai-secrets
                key: endpoint

```

Paso 9. Aplicar el deployment

```

kubectl apply -f rag-deployment.yaml
kubectl get pods -w # Observar el arranque

```

Ejercicio 2.2: Cada estudiante actualiza la imagen a v1.1 (solo cambio de variable de entorno) y ejecuta un rolling update sin downtime: `kubectl set image deployment/rag-function rag-function=acrtechcorp.azurecr.io/rag-function:v1.1`

Capítulo 3: Arquitecturas Serverless

Dominio cognitivo: Aplicar | Sesiones 3.1 y 3.2 | 5 horas totales

Sesión 3.1 — Conceptos Serverless en Azure

Para el chatbot TechCorp adoptamos una arquitectura híbrida: AKS para el orquestador RAG (estado largo, alta CPU) y Azure Functions serverless para los triggers de eventos (nuevo ticket, notificación, resumen diario).

Criterio	AKS (contenedor)	Azure Functions (serverless)
Arranque en frío	~10s	< 500ms (plan Flex Consumption)
Costo en idle	Siempre activo (VM)	Cero (pago por invocación)
Ejecución máxima	Sin límite	10 minutos por defecto

Criterio	AKS (contenedor)	Azure Functions (serverless)
Uso ideal	Flujo RAG principal	Webhooks, notificaciones, resumen

Sesión 3.2 — Práctica: Función Serverless + LLM

Crear la función de resumen diario de tickets

Paso 10. Crear el Function App en Azure

```
az storage account create \
  --name stgtechcorpfn \
  --resource-group rg-techcorp-llm \
  --sku Standard_LRS

az functionapp create \
  --resource-group rg-techcorp-llm \
  --consumption-plan-location eastus2 \
  --runtime python \
  --runtime-version 3.11 \
  --functions-version 4 \
  --name fn-techcorp-summary \
  --storage-account stgtechcorpfn
```

Paso 11. Código de la función (function_app.py)

```
import azure.functions as func
from openai import AzureOpenAI
import logging, os

app = func.FunctionApp()
client = AzureOpenAI(
    azure_endpoint=os.getenv('AZURE_OPENAI_ENDPOINT'),
    api_key=os.getenv('AZURE_OPENAI_KEY'),
    api_version='2024-08-01-preview'
)

@app.timer_trigger(schedule='0 0 18 * * 1-5',
                  arg_name='timer',
                  run_on_startup=False)
def daily_summary(timer: func.TimerRequest) -> None:
    tickets = fetch_open_tickets() # Cosmos DB query
    response = client.chat.completions.create(
        model='gpt-4o',
        messages=[{
            'role': 'system',
```

```
        'content': 'Eres analista de soporte IT. Resume los tickets del día.'
    }, {
        'role': 'user',
        'content': f'Tickets abiertos: {tickets}'
    }],
    max_tokens=500
)

send_email_summary(response.choices[0].message.content)
logging.info('Resumen enviado correctamente.')
```

Paso 12. Desplegar la función

```
func azure functionapp publish fn-techcorp-summary
```

Ejercicio 3.2: Modificar el trigger de Timer a un HTTP trigger para probarlo manualmente desde el portal Azure. Verificar el resultado en Application Insights > Live Metrics.

Capítulo 4: Optimización de Costos

Dominio cognitivo: Aplicar | Sesiones 4.1 y 4.2 | 5 horas totales

Sesión 4.1 — Estrategias de Reducción de Costos

Estimación de costos mensual para TechCorp (sin optimizar)

Servicio	Costo/mes (USD)	Observación
AKS (2x Standard_D4s_v3)	\$280	Nodos activos 24/7
Azure OpenAI GPT-4o	\$1,200	500k tokens/día estimados
Azure AI Search (S1)	\$245	Con vectores
Azure Functions	\$12	Plan consumo
Cosmos DB (10 GB)	\$25	Modo serverless
Ancho de banda + otros	\$80	Estimado
TOTAL SIN OPTIMIZAR	\$1,842	Bajo el presupuesto de \$2,500

- Con las siguientes optimizaciones se puede reducir a USD \$1,100-1,300/mes:
- AKS con spot node pools para cargas no críticas (procesamiento batch): ahorro del 60-80% en esos nodos.
 - Azure OpenAI con caché semántica (Azure API Management + Redis): las preguntas frecuentes reutilizan respuestas previas.
 - Reserved Instances de 1 año para los nodos AKS de producción: hasta 40% de descuento.

- Escalar a cero los nodos fuera de horario de oficina (20:00 – 06:00 LT): reducción adicional del 30%.

Paso 13. Agregar un spot node pool al clúster AKS

```
az aks nodepool add \
  --resource-group rg-techcorp-llm \
  --cluster-name aks-techcorp \
  --name spotpool \
  --priority Spot \
  --eviction-policy Delete \
  --spot-max-price -1 \
  --node-count 2 \
  --node-vm-size Standard_D4s_v3
```

Sesión 4.2 — Práctica de Optimización

Configurar alertas de costos y análisis

Paso 14. Crear un presupuesto con alerta automática (Azure CLI)

```
az consumption budget create \
  --budget-name budget-techcorp-llm \
  --amount 2000 \
  --time-grain Monthly \
  --start-date 2024-12-01 \
  --end-date 2025-12-31 \
  --resource-group rg-techcorp-llm \
  --threshold 80 \
  --contact-emails arquitecto@techcorp.com
```

Paso 15. Activar caché semántica con Redis + APIM

```
# Crear Azure Cache for Redis (C0 = tier gratuito para pruebas)
az redis create \
  --name redis-techcorp \
  --resource-group rg-techcorp-llm \
  --location eastus2 \
  --sku Basic \
  --vm-size c0
```

En APIM se configura una política XML de caché que hasha el prompt del usuario y retorna la respuesta cacheada si el coseno de similitud supera 0.95 (Azure API Management Semantic Caching Policy, disponible desde 2024).

Ejercicio 4.2: Usar Azure Cost Analysis (portal) para filtrar costos por etiqueta 'env:prod' vs 'env:dev'. Calcular el ROI de la caché semántica asumiendo que el 40% de las consultas son repetitivas.

Capítulo 5: Reducción de Latencia y Alta Disponibilidad

Dominio cognitivo: Aplicar | Sesiones 5.1 y 5.2 | 5 horas totales

Sesión 5.1 — Técnicas de Mejora de Rendimiento

Balanceo de carga y autoescalado para el chatbot TechCorp

La carga del chatbot no es uniforme: hay un pico entre las 8:00 y las 10:00 (entrada al trabajo) y otro entre las 13:00 y las 14:00 (regreso del almuerzo). Kube HPA (Horizontal Pod Autoscaler) responde a métricas de CPU y peticiones por segundo.

Paso 16. Configurar HPA basado en CPU y solicitudes HTTP

```
kubectl autoscale deployment rag-function \
  --cpu-percent=60 \
  --min=2 \
  --max=10

# Verificar el HPA
kubectl get hpa rag-function --watch
```

Paso 17. Agregar KEDA para escalar según cola de mensajes Azure Service Bus

```
helm repo add kedacore https://kedacore.github.io/charts
helm install keda kedacore/keda \
  --namespace keda --create-namespace

# ScaledObject que escala pods según mensajes pendientes en Service Bus
kubectl apply -f - <<EOF
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: rag-scaledobject
spec:
  scaleTargetRef:
    name: rag-function
  minReplicaCount: 1
  maxReplicaCount: 20
  triggers:
  - type: azure-servicebus
    metadata:
      queueName: tickets-queue
      messageCount: '5'
EOF
```

Paso 18. Configurar Azure Front Door para distribución geográfica

```
az afd profile create \
  --profile-name afd-techcorp \
  --resource-group rg-techcorp-llm \
  --sku Standard_AzureFrontDoor

az afd origin-group create \
  --profile-name afd-techcorp \
  --origin-group-name og-aks \
  --resource-group rg-techcorp-llm \
  --probe-request-type GET \
  --probe-protocol Https \
  --probe-interval-in-seconds 30 \
  --sample-size 4 \
  --successful-samples-required 3
```

Sesión 5.2 — Pruebas de Carga y Rendimiento

Simular 1,000 usuarios concurrentes con k6

Paso 19. Instalar k6 y crear el script de prueba de carga

```
sudo gpg -k
sudo gpg --no-default-keyring \
  --keyring /usr/share/keyrings/k6-archive-keyring.gpg \
  --keyserver hkps://keyserver.ubuntu.com:80 \
  --recv-keys C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo 'deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] https://dl.k6.io/deb
stable main' \
  | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt-get update && sudo apt-get install k6
```

Script de prueba (load_test.js):

```
import http from 'k6/http';
import { sleep, check } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // Rampa a 100 usuarios
    { duration: '5m', target: 1000 }, // Pico de 1,000 usuarios
    { duration: '2m', target: 0 }, // Bajada gradual
  ],
  thresholds: {
    http_req_duration: ['p(95)<2000'], // SLA: p95 < 2s
    http_req_failed: ['rate<0.01'], // Error rate < 1%
```

```
    },
  };

export default function () {
  const payload = JSON.stringify({
    question: '¿Cómo configuro la VPN en Windows 11?'
  });
  const res = http.post(
    'https://api-techcorp.azure-api.net/chat',
    payload,
    { headers: { 'Content-Type': 'application/json' } }
  );
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}
```

Paso 20. Ejecutar la prueba

```
k6 run load_test.js --out json=results.json
```

Análisis esperado: con 2 réplicas AKS el p95 supera los 3s en el pico. Con HPA activado, escala a 6 réplicas y el p95 baja a 1.4s. Los estudiantes documentan esta curva en su informe de la sesión.

Capítulo 6: Proyecto — Integración Completa

Dominio cognitivo: Crear | Sesiones 6.1 y 6.2 | 5 horas totales

Sesión 6.1 — Diseño de la Infraestructura

Diagrama C4 — Nivel 2: Contenedores de TechCorp Chatbot

El diagrama C4 Nivel 2 (Containers) muestra los siguientes bloques y relaciones:

Bloque	Tecnología Azure	Comunicación
Web App	Azure Static Web Apps	HTTPS → APIM
API Gateway	Azure API Management	JWT + HTTPS → AKS
RAG Orchestrator	AKS (Python FastAPI)	HTTPS → AI Search + OpenAI
Vector Search	Azure AI Search	REST API
LLM	Azure OpenAI (GPT-4o)	SDK Python
Historial	Azure Cosmos DB	SDK Python async
Observabilidad	Azure Monitor + AppInsights	OpenTelemetry SDK

Paso 21. Indexar la base de conocimiento IT en Azure AI Search

```
az search service create \
  --name srch-techcorp \
  --resource-group rg-techcorp-llm \
  --sku Standard \
  --partition-count 1 \
  --replica-count 1

# El indexador vectorial se configura desde Python:
from azure.search.documents.indexes import SearchIndexClient
from azure.search.documents.indexes.models import (
    SearchIndex, SimpleField, SearchableField,
    VectorSearch, HnswAlgorithmConfiguration,
    SearchField, SearchFieldDataType
)

index = SearchIndex(
    name='knowledge-base',
    fields=[
        SimpleField(name='id', type='Edm.String', key=True),
        SearchableField(name='content', type='Edm.String'),
        SearchField(
            name='contentVector',
            type=SearchFieldDataType.Collection(SearchFieldDataType.Single),
            searchable=True,
            vector_search_dimensions=1536,
            vector_search_profile_name='myHnswProfile'
        )
    ],
    vector_search=VectorSearch(
        profiles=[VectorSearchProfile(name='myHnswProfile',
            algorithm_configuration_name='myHnsw')],
        algorithms=[HnswAlgorithmConfiguration(name='myHnsw')]
    )
)

client.create_index(index)
```

Sesión 6.2 — Implementación y Pruebas Finales

Pipeline CI/CD completo con GitHub Actions + Azure

Paso 22. Crear el workflow de despliegue continuo

Archivo .github/workflows/deploy.yml:

```
name: Deploy TechCorp Chatbot
on:
```

```
push:
  branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to ACR
        uses: azure/docker-login@v1
        with:
          login-server: acrtechcorp.azurecr.io
          username: ${ secrets.ACR_USERNAME }
          password: ${ secrets.ACR_PASSWORD }

      - name: Build and push
        run: |
          docker build -t acrtechcorp.azurecr.io/rag-function:${ github.sha } .
          docker push acrtechcorp.azurecr.io/rag-function:${ github.sha }

      - name: Set AKS context
        uses: azure/aks-set-context@v3
        with:
          resource-group: rg-techcorp-llm
          cluster-name: aks-techcorp

      - name: Rolling update
        run: |
          kubectl set image deployment/rag-function \
            rag-function=acrtechcorp.azurecr.io/rag-function:${ github.sha }
          kubectl rollout status deployment/rag-function
```

Paso 23. Validación final de la arquitectura

1. Ejecutar prueba de carga final con k6 (1,000 usuarios, 10 minutos).
2. Verificar que el p95 de latencia es < 2s en Azure Monitor > Workbooks.
3. Confirmar que el HPA escaló correctamente (mínimo 4 réplicas en el pico).
4. Revisar costos proyectados en Azure Cost Analysis; deben ser < \$1,500/mes con cache activa.
5. Presentar el diagrama C4 actualizado con las decisiones de diseño (ADR documentado).

Resumen: Mapa de Herramientas por Capítulo

Cap.	Tema	Herramientas Azure	Entregable del alumno	#
1	Conceptos e infraestructura	Azure Portal, Cost Calculator, Azure OpenAI Playground	Documento de requisitos del caso propio	1
2	Docker y Kubernetes	ACR, AKS, kubectl, Azure CLI	Imagen Docker + Deployment YAML funcional	2
3	Serverless	Azure Functions, APIM, Service Bus	Función serverless integrada con GPT-4o	3
4	Costos	Cost Analysis, Reservations, Redis Cache, Spot Pools	Informe de optimización con ahorro estimado	4
5	Rendimiento y HA	HPA, KEDA, Front Door, k6	Reporte de prueba de carga con thresholds SLA	5
6	Proyecto integrador	AI Search, Cosmos DB, GitHub Actions, OpenTelemetry	Arquitectura completa desplegada y documentada	6

Nota del Arquitecto: Todos los recursos creados en este curso caben dentro del crédito de Azure for Students (USD \$100). Se recomienda destruir los recursos al finalizar cada capítulo con 'az group delete --name rg-techcorp-llm --yes --no-wait' para evitar costos inesperados.